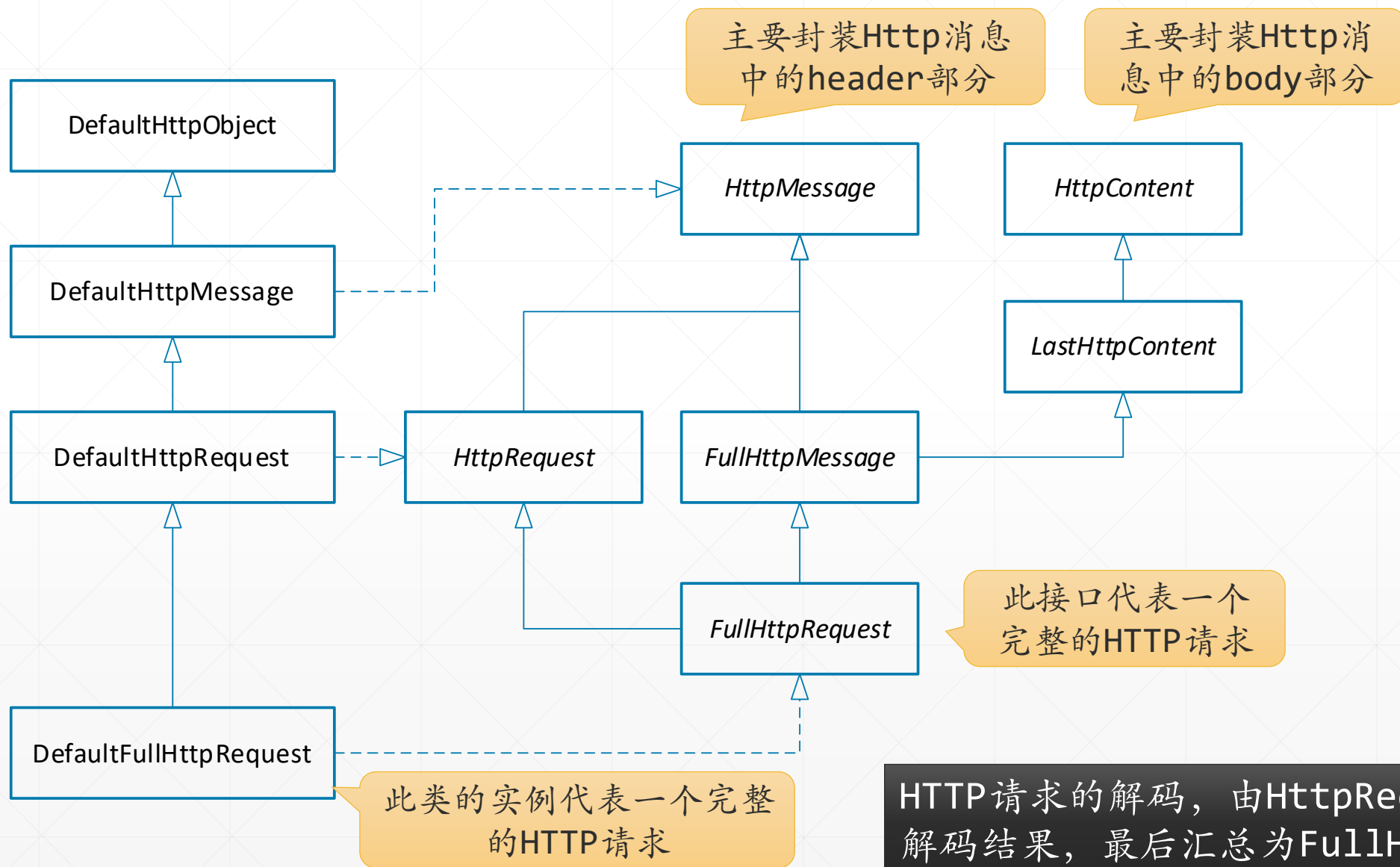


Netty welcome Web!

北京理工大学计算机学院
金旭亮

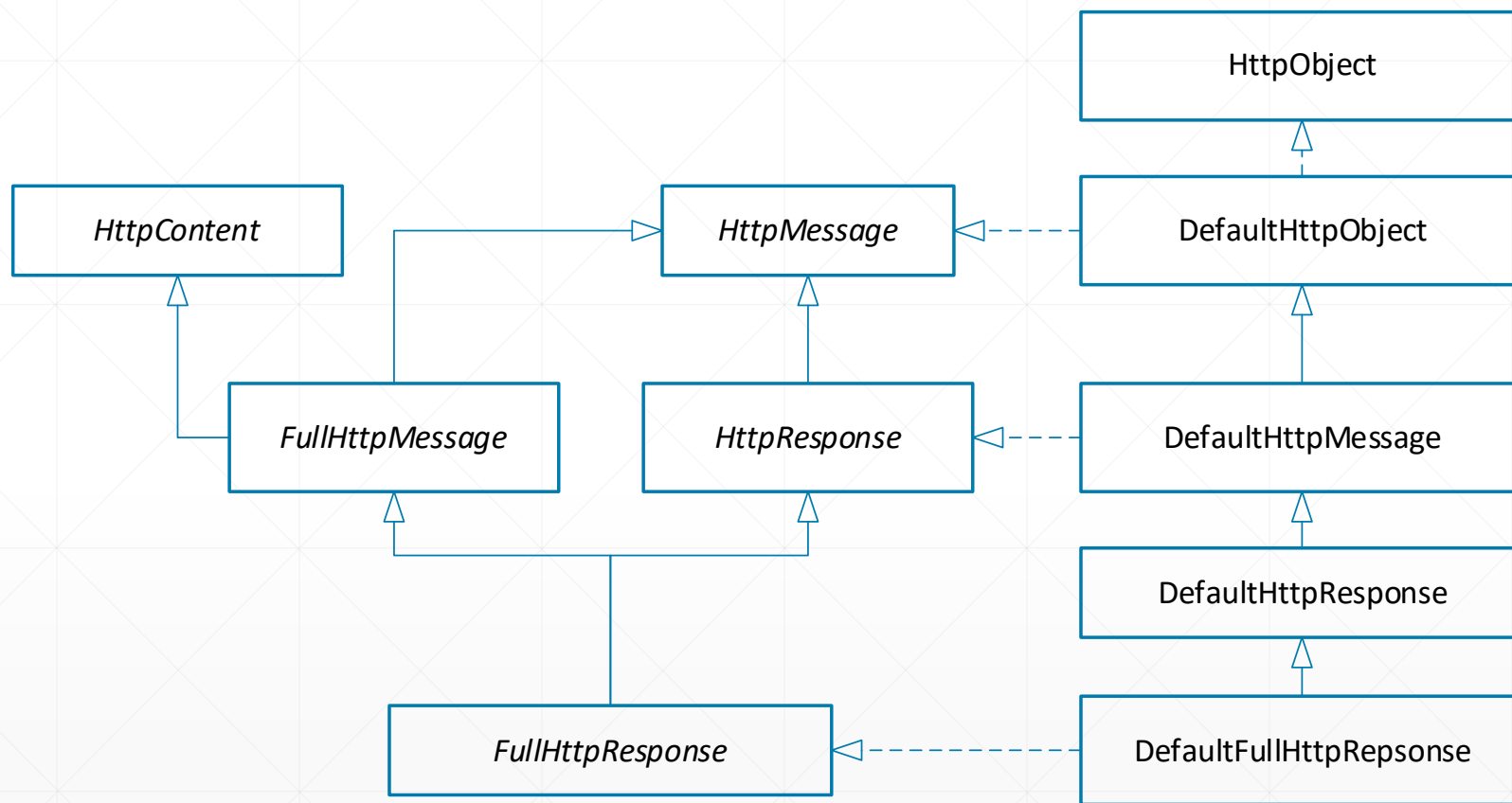


HTTP请求解码相关类型



HTTP请求的解码，由HttpRequestDecoder负责，解码结果，最后汇总为FullHttpRequest。

HTTP响应相关类型



HttpResponseEncoder 负责把HttpResponse和HttpContent编码，并写入到ByteBuf中。

真正用到的实例

内置的编解码Handler

`HttpServerCodec = HttpRequestDecoder + HttpResponseEncoder`



将放在ByteBuf中的数据，解码为HttpRequest和HttpContent接口对象（有一个content字段，引用一个保存了HTTP消息body部分中数据的ByteBuf）。



将HttpResponse和HttpContent接口对象，编码为ByteBuf。

`HttpObjectAggregator = HttpContent + HttpMessage`

最终得到一个FullHttpRequest对象，然后，我们就可以编写业务Handler，从这个请求对象中提取数据，完成业务处理功能。

服务器应用入口点

```
public class HelloNettyServer {  
    public static void main(String[] args) throws InterruptedException {  
  
        try (EventLoopGroup bossGroup = new NioEventLoopGroup(); //处理连接  
            //完成与客户端之间的数据传送与接收  
            EventLoopGroup workerGroup = new NioEventLoopGroup()) {  
            //设置Server端应用所使用的线程组,指定要使用的接收客户端连接的Channel类型  
            //handler对象,则用于与客户端之间进行数据通讯,它实际上是一个由多个Handler构成的管线  
            ServerBootstrap serverBootstrap = new ServerBootstrap();  
            serverBootstrap.group(bossGroup, workerGroup)  
                .channel(NioServerSocketChannel.class)  
                .childHandler(new HelloServerInitializer());  
            //绑定端口并启动  
            ChannelFuture channelFuture = serverBootstrap.bind(inetPort: 8088).sync();  
            channelFuture.channel().closeFuture().sync();  
        }  
    }  
}
```

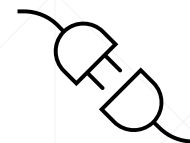
构建HTTP请求处理管线

```
//这个类的主要职责，是为特定的Channel，构建由多个Handler组成的Pipeline
//注意：其基类ChannelInitializer实现了ChannelInboundHandler接口，
//所以，它是一个自己也是一个Handler
1 usage
public class HelloServerInitializer extends ChannelInitializer<SocketChannel> {
    2 usages
    @Override
    protected void initChannel(SocketChannel channel) throws Exception {
        ChannelPipeline pipeline=channel.pipeline();
        //HttpServerCodec是一个handler，它包容两个功能：
        // HttpRequestDecoder 和 HttpResponseEncoder
        pipeline.addLast( name: "HttpServerCodec",new HttpServerCodec());
        //自定义一个Handler
        pipeline.addLast( name: "customHandler",new CustomHandler());
    }
}
```

Netty提供的HttpServerCodec，包含对HTTP请求与响应消息的编码和解码功能，它能够将收到的消息，解码为一个HttpObject对象，然后，后继的业务Handler，就可以使用它来提取相应信息。

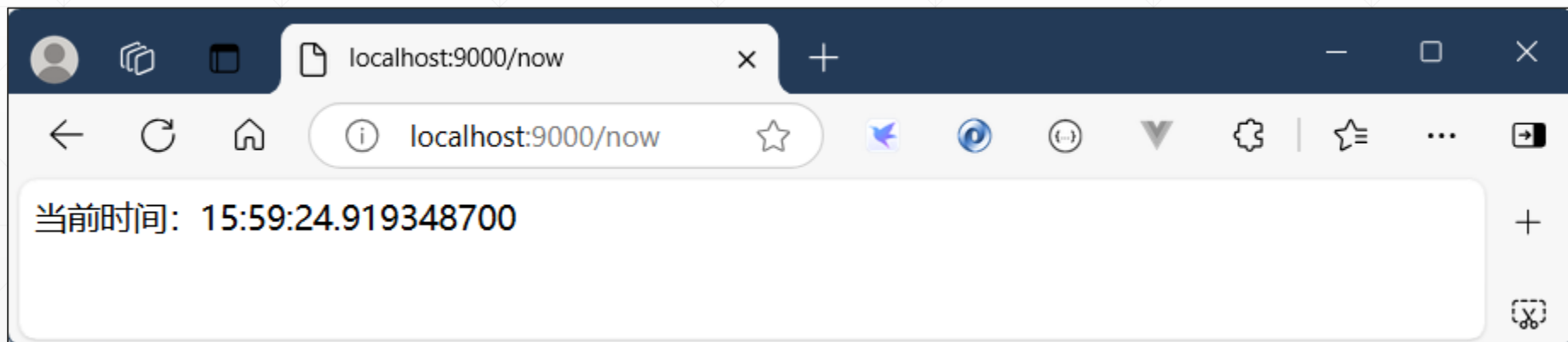
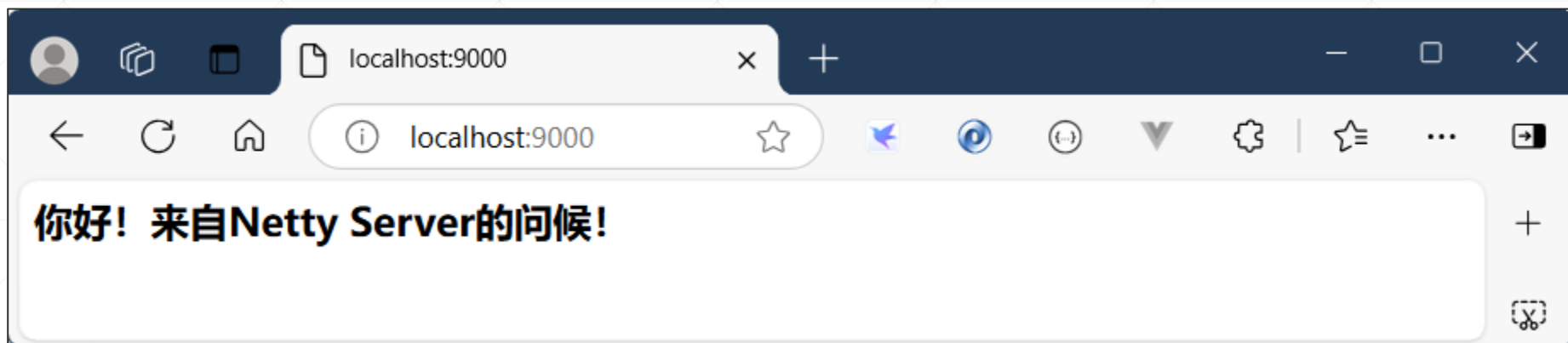
另外一种常见的pipeline构建方式，是在HttpServerCodec之后添加一个HttpObjectAggregator处理器，后面的业务处理器，则从FullHttpRequest对象中提取信息。

编写Handler



```
public class MyHttpHandler extends SimpleChannelInboundHandler<HttpObject> {
    @Override
    protected void channelRead0(ChannelHandlerContext ctx, HttpObject msg) throws Exception {
        //如果收到的消息对象（从前一个Handler传来的）是一个请求对象
        if (msg instanceof HttpRequest request) {
            String uri = request.uri();
            System.out.println("收到的HTTP请求的URL:" + uri);
            String reponseText = "";
            if (uri.equals("/")) {
                reponseText = "<h3>你好! 来自Netty Server的问候! </h3>";
            } else if (uri.equals("/now")) {
                reponseText = "当前时间: " + LocalTime.now();
            }
            ByteBuf content = Unpooled.copiedBuffer(reponseText, CharsetUtil.UTF_8);
            //生成HTTP响应
            FullHttpResponse response = new DefaultFullHttpResponse(HttpVersion.HTTP_1_1, HttpResponseStatus.OK);
            //设置必须的HTTP Header值
            response.headers().set(HttpHeaderNames.CONTENT_TYPE, "text/html;charset=UTF-8");
            response.headers().set(HttpHeaderNames.CONTENT_LENGTH, content.readableBytes());
            //设置HTTP响应的body值
            response.content().writeBytes(content);
            //将响应发回给客户端
            ctx.writeAndFlush(response);
        }
    }
}
```

测试



启用压缩功能



```
boot.group(boosGroup,workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        @Override
        protected void initChannel(SocketChannel socketChannel) throws Exception {
            //启用接收数据的解压功能
            socketChannel.pipeline().addLast(new HttpContentDecompressor());
            //这个中间件，包容了两个子Handler: HttpRequestDecoder 和 HttpResponseEncoder
            //主要完成的，就是编解码功能
            socketChannel.pipeline().addLast(new HttpServerCodec());
            //自定义中间件，用于生成HTTP响应
            socketChannel.pipeline().addLast(new MyHttpHandler());
            //启用发送数据的压缩功能
            socketChannel.pipeline().addFirst(new HttpContentCompressor());
        }
    });
```

Netty提供了HttpContentDecompressor和HttpContentCompressor两个内置Handler，可以完成HTTP消息的压缩与解压功能，但要注意其在管线中的位置。